Waterloo Rocketry 2025 - Aurora

Roll Control Design and Simulation of a Canard-controlled Sounding Rocket

Finn Breuer [1] Joe Dolina [2] Tristan Yan-Klassen [3]

January 2, 2025

[1]Technische Universität Hamburg, finn.breuer@tuhh.de
[2]University of Waterloo
[3]University of Waterloo

Contents

I Estimator & Controller **2**

1 Modelling . . . 2
- 1.1 State Space representation . . . 2
- 1.2 Kinematics and dynamics . . . 3
- 1.3 Barometer, Atmosphere and air data . . . 5
- 1.4 Forces and moments . . . 6
- 1.5 IMU Measurements . . . 7
- 1.6 The models . . . 9

2 Estimator . . . 10
- 2.1 Filter algorithm . . . 10
- 2.2 Implementation . . . 11
- 2.3 Optimal estimation theory . . . 12
- 2.4 Filter tuning . . . 13
- 2.5 Initialization routine . . . 13
- 2.6 Hard calibration . . . 13

3 Controller . . . 14
- 3.1 Controller roll model . . . 14
- 3.2 Feedback law and tuning . . . 15
- 3.3 Gain scheduling . . . 15
- 3.4 Optimal control theory . . . 16

4 Appendix . . . 17
- 4.1 Numerical Jacobian computation . . . 17
- 4.2 Numerical Integration methods . . . 17

5 Index . . . 18

II Flight Simulation **20**

6 Plant Model . . . 20
- 6.1 Background . . . 20
- 6.2 Theory . . . 21
- 6.3 Implementation . . . 21

7 Simulation results . . . 24
- 7.1 Rocket flight . . . 24
- 7.2 Estimator in the loop . . . 24
- 7.3 Controller in the loop . . . 24
- 7.4 Full closed-loop . . . 24

References **25**

Chapter I

Estimator and Controller Design

Finn Breuer

1 Modelling

1.1 State Space representation

Dynamical models of the rocket flight are needed for both the controller synthesis and the estimator algorithm. The rocket model is made up of algebraic and differential equations [1, 2], represented in *state-space* as [3, 2]

$$\dot{x} = f(t, x, u) \tag{I.1}$$
$$y = h(t, x, u) \tag{I.2}$$

with the state x, measurement output y, and control input u. Here the state equation f and the output equation h are nonlinear functions. Linear time-invariant state-space models can be derived by linearizing these equations around some point (t_0, x_0, u_0), which results in the equations

$$\dot{x} = A(x - x_0) + B(u - u_0) \tag{I.3}$$
$$y = C(x - x_0) + D(u - u_0) \tag{I.4}$$

States x

From textbooks [1, 2, 4], lectures [5], papers [6, 7], the industry standard for choosing states of aeronautical vehicles is using attitude angles, attitude rates, velocity, and position. Position is reduced to altitude only (longitude and latitude are not required [1]). The entire state vector is then

$$x = \begin{bmatrix} q & \omega & v & l & C_L & \delta \end{bmatrix}^T \tag{I.5}$$

with the attitude represented as quaternions $q = \begin{bmatrix} q_w & q_x & q_y & q_z \end{bmatrix}^T$, the angular velocity in the body-fixed frame $\omega = \begin{bmatrix} \omega_x & \omega_y & \omega_z \end{bmatrix}^T$ (body rates)[2], the velocity in the body-fixed

[1] and probably undesired as it borders on guided missile design

[2] In literature the body rates are often depicted as $\omega_x = p$, $\omega_y = q$, and $\omega_z = r$. As this may be confusing for people unfamiliar, the rates are only called ω_i in this documentation.

frame $v = \begin{bmatrix} v_x & v_y & v_z \end{bmatrix}^T$, and the altitude over sea level l. Additionally, the canards have the coefficient of lift C_L as we are unsure about the aerodynamic behaviour of the canards (and as a state, the coefficient can be estimated), and the angle-of-attack δ which is used to carry first order actuator dynamics.

Measurements y

The measurements arrive in the measurement vector

$$y = \begin{bmatrix} \Omega & M & P & T \end{bmatrix}^T \tag{I.6}$$

with measured body rates from the rate gyroscope Ω, the magnetic field measurement M (which can be transformed to attitude [3]) from the magnetometer, additionally the static air pressure P, and temperature T are provided by the barometer. The accelerometer data A will be provided to the filter in the input vector, as this simplifies calculations.[4] If multiple sensors are used for each measurement, they will be stacked in the vector as $y = \begin{bmatrix} \Omega_1 & \Omega_2 & M_1 & M_2 & P_1 & P_2 & T_1 & T_2 \end{bmatrix}$.

Inputs u

The control input vector is the desired canard angle to the servo mechanism δ_u and, as stated in the previous paragraph, the accelerometer data A, which may also be stacked.

$$u = \begin{bmatrix} \delta_u & A \end{bmatrix}^T \tag{I.7}$$

The actual canard angle is represented as the state δ, to account for actuator dynamics. The computation from desired canard angle to servo command should be performed in the controller.

1.2 Kinematics and dynamics

Coordinate frames

The inertial frame f^G is assumed to be the earth-flat geographic frame, with an up-north-west orientation. The body-fixed frame f^B is oriented along the main inertia axes of the rocket body, namely forward-rail-port (x^+ is forward). The sensor frame f^S is aboard the rocket, but rotated and translated against the body frame. The rotational transformation between the frames is given by

$$r^G = S r^B \qquad r^B = S^T r^G \tag{I.8}$$

$$r^B = S_S r^S \qquad r^S = S_S^T r^B \tag{I.9}$$

with the rotation matrix S (see Equation I.13), and the rotation matrix S_S for the sensor frame. The frames f^G and f^B are shown in Figure I.1 with their rotation against each other. To simplify the following notations, the velocities, the angular rates, and the external forces and torques are given in the body-fixed frame, and the index ^{B} is omitted.

[3] The inverse transformation is not unique as it is invariant about the field direction, but we can't use the accelerometer in free flight to find the gravity vector

[4] This is quite common practice

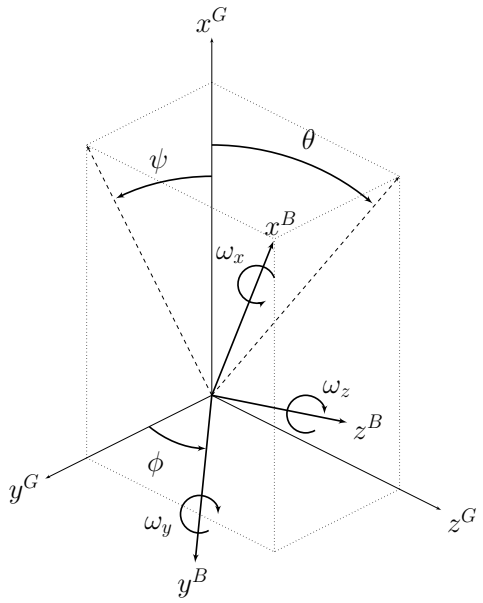


Figure I.1: Diagram of the attitude coordinate frames with Euler angles and body rates.

Attitude with Quaternions

> "These differential equations are a joy to implement, because they are linear, have no singularities, and number only four" - P. Zipfel [1]

The Quaternion is defined as

$$q = \begin{bmatrix} q_w & q_x & q_y & q_z \end{bmatrix}^T = \begin{bmatrix} q_w \\ \boldsymbol{q} \end{bmatrix} \tag{I.10}$$

which is normed to $|q| = q^T q = 1$ to describe rotations. This has to be enforced in the filter process, by norming q after every integration. The quaternion derivative is [1, 8]

$$\begin{bmatrix} \dot{q}_w \\ \dot{\boldsymbol{q}} \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 0 & -\omega^T \\ \omega & -\tilde{\omega} \end{bmatrix} \cdot \begin{bmatrix} q_w \\ \boldsymbol{q} \end{bmatrix} = \frac{1}{2} \begin{bmatrix} q_w & \boldsymbol{q}^T \\ -\boldsymbol{q} & q_w \boldsymbol{I} + \tilde{\boldsymbol{q}} \end{bmatrix} \cdot \begin{bmatrix} 0 \\ \omega \end{bmatrix} \tag{I.11}$$

with angular rates ω described in the body-fixed frame, and

$$\tilde{\omega} = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix} \qquad \tilde{\boldsymbol{q}} = \begin{bmatrix} 0 & -q_z & q_y \\ q_z & 0 & -q_x \\ -q_y & q_x & 0 \end{bmatrix} \tag{I.12}$$

The rotation matrix, used for attitude transformation from the body-fixed frame to the inertial frame, can be computed directly from the quaternion [1, 8] as

$$S = I + 2q_w \tilde{\boldsymbol{q}} + 2\tilde{\boldsymbol{q}}\tilde{\boldsymbol{q}} \tag{I.13}$$

The Euler angles *yaw* ψ, *pitch* θ, and *roll* ϕ (in airplanes heading, pitch, bank) can be recovered from the rotation matrix (or directly from the quaternions). For the rotation sequence yaw-pitch-roll the Euler angles are

$$\psi = \operatorname{atan2}(S_{21}, S_{11}) \qquad \theta = \arcsin(-S_{31}) \qquad \phi = \operatorname{atan2}(S_{23}, S_{33}) \tag{I.14}$$

For initialization, the quaternion is set as

$$q_w = \cos(\frac{\varphi}{2}) \qquad \boldsymbol{q} = \boldsymbol{d}\sin(\frac{\varphi}{2}) \tag{I.15}$$

where φ is a rotation about a unit vector $\boldsymbol{d}$, both of which are set according to the initial orientation of the rocket [5].

Relative motion

Using Newton's second law, the acceleration in the body-fixed frame is [1, 8]

$$\dot{v} = m^{-1}\mathcal{F} - 2\,\omega \times v \tag{I.16}$$

which includes external forces $\mathcal{F}$ (described in the body-fixed frame) and the tangent acceleration. The velocity in the inertial (geographic) frame is [1]

$$\dot{r}^G = Sv \tag{I.17}$$

With Euler's law, the angular acceleration is [1, 8]

$$\dot{\omega} = J^{-1}(\mathcal{T} - \omega \times J\omega) \tag{I.18}$$

which includes the external torques (or moments) $\mathcal{T}$ (described in the body-fixed frame) and the gyroscopic moment. The inertia tensor J is determined from CAD or experimentally. If the coupling inertia moments are small, they may be neglected to fill the tensor diagonally as $J = \text{diag}\begin{bmatrix} J_x & J_y & J_z \end{bmatrix}$, where in a rocket cruciform often $J_y = J_z$.

1.3 Barometer, Atmosphere and air data

The air at altitude l is modelled according to the US standard atmosphere [9]

$$T = T_B - k_B(l - l_B) \tag{I.19}$$

$$p = p_B \exp\left(\frac{-g(l - l_B)}{RT_B}\right) \qquad p = p_B\left(\frac{T_B}{k_B(l - l_B)}\right)^{\frac{-g}{Rk_B}} \tag{I.20}$$

$$\rho = \frac{p}{RT} \tag{I.21}$$

$$c = \sqrt{\gamma RT} \tag{I.22}$$

with temperature T, static pressure p, density ρ, and the local speed of sound c. where the index B indicates a base value from Table I.1. The adiabatic index of air is $\gamma = 1.4$, and the specific gas constant of air is $R = 287.0579\frac{\text{J}}{\text{kgK}}$. To calculate the static air pressure, Equation I.20 is split in two, where the left equation is only used for a zero lapse rate $k_B = 0$. The mach number is $\text{Ma} = V_A/c$. The dynamic pressure is defined with the airspeed $V_A = |v|$ (assuming no or only little wind disturbances [6]) as [9]

$$\bar{p} = \frac{1}{2}\rho V_A^2 \tag{I.23}$$

[5] Initialization with Euler angles is also possible [1, p. 126], but too much effort to write down here.
[6] These are difficult to predict, and will thus act as disturbances on the rocket-estimator loop

Layer	Altitude l_B [m]	Pressure p_B [Pa]	Temperature T_B [K]	Lapse rate k_B [$\frac{\mathrm{K}}{\mathrm{m}}$]
Troposphere	0	101325	288.15	0.0065
Tropopause	11000	22632.1	216.65	0
Stratosphere	20000	5474.9	216.65	-0.001
Stratopause	32000	868.02	228.65	-0.0028

Table I.1: Atmosphere base data

1.4 Forces and moments

Gravity

Gravitational acceleration acts in the geographic down direction, so the gravitational force in the body-fixed frame is

$$\mathcal{F}_G = -mS^T g^G \tag{I.24}$$

Aerodynamic forces and moments

Most of the external forces and torques acting on the rocket will stem from aerodynamics. The effect of rocket body geometry, including fins and nosecone, can be approximated using Barrowman's method [10].

The lift force for each component is defined as

$$\mathcal{F}_L = \bar{p}\, C_L A \tag{I.25}$$

where the coefficient of lift C_L and reference area A depend on the geometry of the component, and the lift coefficient encompasses information about the angle of attack.

The torque produced by the aerodynamic forces is determined by the location of the *center of pressure* and acts about the center of gravity as

$$\mathcal{T}^i = \mathcal{F}^i \times d^i \tag{I.26}$$

where d^i is the vector from the center of pressure to the center of gravity (direction matters!). Here ^{i} indicates an arbitrary component (it is not an exponent).

Aerodynamics of the canards

The coefficient of lift is approximated using thin airfoil theory [9]. The canards have a low aspect ratio of about $0.5 \lesssim AR \lesssim 1$, and thin airfoil theory thus predicts a lift coefficient for each canard of [9]

$$C_L^{\mathrm{can}} = \frac{\pi}{2} AR\, \delta \approx 1.5\, \delta \tag{I.27}$$

with the canard angle of attack δ. As the canards are of equal size, and move in equal and opposite angles, the normal force on the rocket body cancels out (for small angles of attack of the rocket body), it remains a net moment about the rocket x-axis. The forces and moments produced by the canards are thus

$$\mathcal{F}^{\mathrm{can}} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \qquad \mathcal{T}^{\mathrm{can}} = \begin{bmatrix} \bar{p}\, C_{L_\delta}^{\mathrm{can}} \delta A^{\mathrm{can}} d_y^{\mathrm{can}} \\ 0 \\ 0 \end{bmatrix} \tag{I.28}$$

where A^{can} is the total area of the canards and d_y^{can} is the distance between the rocket roll axis and the center of pressure of one of the canards. The instantaneous roll acceleration can be written with the roll control derivative L_δ as

$$\dot{\omega}_x = \mathcal{T}_x^{\text{can}}/J_x = L_\delta\,\delta \qquad\qquad L_\delta = \bar{p}\,C_{L_\delta}^{\text{can}} A^{\text{can}} d_y^{\text{can}}/J_x \tag{I.29}$$

Aerodynamics of the rocket body

Propulsion

As we cannot predict thrust misalignment, it is assumed zero with any misalignment acting as disturbances. Any sloshing or pogo-effect is too complex to model here, and will be neglected.

Rail constraints

The rocket is attached to the launch rail for the first few meters of altitude, constraining the rocket to one degree of freedom (translating upwards).

1.5 IMU Measurements

The Inertial Measurement Unit (IMU), which houses all sensors, is not mounted at the center of gravity of the rocket body, and the sensor frame is not aligned to the body frame. Therefore, a transformation to the body-fixed frame must be formulated to model the sensor measurements using current states.

Angular velocity

The measured angular velocity Ω is in the rotated sensor frame, and needs to be transformed to the body-fixed frame as[7]

$$\omega = S_\Omega \hat{\Omega} \qquad\qquad \hat{\Omega} = S_\Omega^T \omega \tag{I.30}$$

where S_Ω is the rotation vector from body-fixed to the sensor frame. This transformation is done for each IMU separately.

Acceleration

As the acceleration is not a state, and cannot be directly computed from the other states (aerodynamic modelling is too indirect), it is used in the state prediction step instead of the correction step. The measured acceleration A enters the prediction function as a control input. Transformation from the sensor frame to the body frame are computed as [2] [8]

$$a = S_A A - \dot{\omega} \times r_s - \omega \times (\omega \times r_s) \tag{I.31}$$

resulting in the corrected specific force term a in the body-fixed frame, with the vector from rocket center of mass to the sensor r_s, and the rotation matrix from the sensor frame

[7] In a lot of literature, Ω refers to the skew-symmetric matrix of ω. Here, this matrix is expressed as $\tilde{\omega}$ or $(\omega\times)$, while Ω refers to sensor measurements

[8] I'm about 90% sure that this is the correct transformation and derivation

to the body frame S_A. This transformation is performed for each IMU separately. The resulting acceleration in the body-fixed frame is then [1, 2] [9]

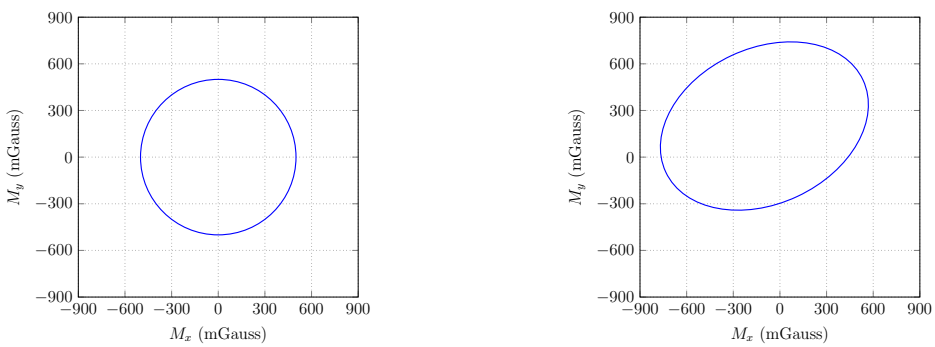
$$\dot{v} = a - 2\,\omega \times v + S^T g^G \tag{I.32}$$

which accounts for the derivation in the rotating body-fixed frame and the gravitational acceleration, which cannot be measured by the accelerometer. As these measurements do not enter the filter in the classical way, their weighting of the expected covariances has to happen in the prediction model. The transformation to the body-fixed body-centered frame from Equation I.31 is performed first, after which the a_i will be weighted and then fed to Equation I.32.

Magnetometer

The magnetometer measures the local magnetic field vector. This is mostly determined by Earths magnetic field M_E (which is assumed to be constant over the flight, and predetermined by initialization on the ground), but can include disturbances from local sources on the rocket (and ground support equipment).

Hard iron disturbances emanate from permanent and electromagnets, which cause an offset bias by vector B_{HI} in the measured field strength as shown in Figure I.2b.
Soft iron disturbances stem from nearby ferromagnetic materials (predominantly Iron, Nickel, Cobalt), which react to the present magnetic field. This causes stretching of the measured field (shown in Figure I.2b) by the matrix B_{SI}, which is unity if no soft iron disturbances are present.



(a) Disturbance free (b) Hard and soft iron disturbances

Figure I.2: 2D Visualization of the measured magnetic field [11]

The model of the measured magnetic field with hard and soft iron biases B_{HI} and B_{SI} is

$$\hat{M} = B_{SI}(S^T S_M^T M_E) + B_{HI} \tag{I.33}$$

where Earths magnetic field vector M_E is rotated into the body-fixed and then the sensor frame with $S^T S_M^T$. Hard calibration (Section 2.6) must account for both biases and sensor frame rotation.

[9] Some literature [1] does not have the factor 2 in the Coriolis term. Unsure why they omitted that, simple mistake?

1.6 The models

The state prediction model

$$\dot{\hat{x}} = f(t, \hat{x}, u) \tag{I.34}$$

predicts the time derivative of the state vector $\dot{\hat{x}}$ (Equation I.5), using the control input u (Equation I.7), and the following model equations:

- Attitude: I.11
- Rates: I.18
- Velocities: I.32 for filtering and I.16 for simulation
- Position/Altitude: I.17
- Canard coefficient: 1st order dynamics $\dot{C}_L = -(1/\tau_c)(C_L - C_{L,\,\text{theory}})$
- Canard angle: 1st order dynamics $\dot{\delta} = -(1/\tau_\delta)(\delta - \delta_u)$

The measurement model

$$\hat{y} = h(t, \hat{x}, u) \tag{I.35}$$

predicts the sensor outputs $\hat{y}$ from the current state estimate $\hat{x}$, and the model equations:

- Gyroscope: I.30
- Accelerometer: used in state prediction as I.31 and I.32
- Magnetometer: I.33
- Barometer: I.19 to I.22

2 Estimator

The state estimation algorithm is responsible for accurately determining the current rocket state. The rocket state is a set of data points which describe the condition of the rocket dynamics, and is made up of the rocket attitude, velocity, altitude, aerodynamic coefficients (coefficients are not quite certain due to the lack of available windtunnel testing) and other air data. As the underlying dynamics are nonlinear in some cases, an *Extended Kalman Filter* (EKF) is used to estimate the current state. The EKF algorithm uses these modelled dynamics to

1. predict the current state from the last state and control inputs (desired canard angles) to the rocket, and then
2. correct the state using measurements from onboard sensors, including rate gyroscopes, magnetometers, accelerometers, barometers.

This whole process is a *Bayes filter*, where the state estimate has an associated belief in the accuracy of the estimate. Each iteration of the filter uses the the priori (old) state estimate and belief, to compute/update the posteriori (new) estimate and belief.

2.1 Filter algorithm

The EKF thus propagates a state estimate $\hat{x}$ in discrete time steps, along with the belief in form of the covariance matrix P. The matrix represents the "uncertainty" of the state estimate and their correlation, i.e. higher values represent more uncertainty.

For its algorithm the EKF uses the continuous-time prediction model f and the discrete-time measurement model h, as well as their Jacobians

$$F(x,u) = \frac{\partial f}{\partial x} \qquad\qquad H(x) = \frac{\partial h}{\partial x} \tag{I.36}$$

which can be understood as the *sensitivity* of the nonlinear equations f and h to variations of the state. Essentially, the Jacobians indicate how much the outputs of the functions f and h change for small changes of each state variable. As the the filter is computed at sufficiently short (see Section 2.2) time intervals, the state does not change greatly between each computation step. The functions f and h can be approximated by linearization, and the covariance is propagated using the resulting Jacobians.

One complete iteration of the algorithm of the Continuous-Discrete EKF is [3, 12, 6] [10]

Predict:

solve IVP $\left(\hat{x}_k^- \ , \ P_k^-\right)$ with $\hat{x}(0) = \hat{x}_{k-1}^+ \ , \ P(0) = P_{k-1}^+$

$$\dot{\hat{x}}(t) = f(\hat{x}, u_{k-1}) \tag{I.37}$$

$$\dot{P}(t) = F(\hat{x}, u_{k-1})P + PF^T(\hat{x}, u_{k-1}) + Q_e \tag{I.38}$$

Correct:

$$K_k = P_k^- H^T(\hat{x}_k^-)\left[H(\hat{x}_k^-)P_k^- H^T(\hat{x}_k^-) + R_e\right]^{-1} \tag{I.39}$$

$$\hat{x}_k^+ = \hat{x}_k^- + K_k\left[y_k - h(\hat{x}_k^-)\right] \tag{I.40}$$

$$P_k^+ = \left[I - K_k H(\hat{x}_k^-)\right] P_k^- \tag{I.41}$$

[10] This is not the cleanest formulation, but hopefully understandable. See [3, pp. 259] for more thorough explanations of the algorithm.

where the measurements are used for the state correction, while the control input is used for prediction.

Here the prediction part consists of the continuous-time dynamics (Equation I.37) and the associated covariance in continuous-time (Equation I.38). These dynamics and the solution of the previous filter iteration ($\hat{x}(0) = \hat{x}_{k-1}^{+}$ and $P(0) = P_{k-1}^{+}$) yield an *Initial Value Problem* (IVP). The discrete solutions $\hat{x}_k^-$ and P_k^- of the IVP have to be computed by integration. As the IVP is solved in each filter iteration, the integration bounds are the timestamps of the previous and current iteration.

The correction part computes the filter gain (or Kalman gain) K_k using the covariance of the estimate P and the sensitivity of the measurement model H (Equation I.39). As the measurement model is discrete, the computation of the predicted measurements ($\hat{y} = h(t, \hat{x}_k^-)$) and the correction of the state estimate (Equation I.40) is straightforward. The covariance of the estimate is updated accordingly (Equation I.41).

The EKF is tuned using the weighting matrices Q_e and R_e, which represented the expected noise covariances of plant and measurement (see Section 2.4).

To check if the filter can converge at a given state, the linearized system is examined for observability. The observability matrix (with the number of states $n = 13$) is

$$\mathcal{O} = \begin{bmatrix} H \\ HF \\ HF^2 \\ \vdots \\ HF^n \end{bmatrix} \tag{I.42}$$

which has rank 12 for the given model, i.e. the system is not fully observable. The reason is the lack of an absolute measurement in attitude, as the magnetometer measurement is invariant for rotations about the magnetic field vector. This will result in slow attitude drift from the true value, which is still an improvement over simply integrating the rate gyroscopes as the magnetometer constrains the attitude in the other two degrees of freedom.[11]

The EKF is initialized by setting the state estimate and the covariance estimate to

$$\hat{x}(0) = \bar{x}_0 \qquad\qquad P(0) = \bar{P}_0 \tag{I.43}$$

where $\bar{x}_0$ can be computed while the rocket is stationary on the rail, and $\bar{P}_0$ is the uncertainty of the initial state (often quite low). Determining accurate initial values is treated in Section 2.5.

2.2 Implementation

The resulting state is passed to the control algorithm.

The Jacobians F and H are computed numerically. Numerical methods for finite differentiation are shown in Section 4.1.

[11] For vehicles in straight and level movement, like ships, cars, and even airplanes, this is corrected by accelerometers measuring the gravity vector. However, the rocket will be in free fall the entire flight and measuring gravity is not possible.

For the prediction step, numerical solver are implemented to compute the solutions of the IVPs. In this case these methods are *Runge-Kutta(4)* for the state prediction and *Heun's method* for the state covariance (see Section 4.2). The methods are picked as a trade-off between accuracy and computational effort. Computation time is an important factor in implementing the state estimation, as the discrete EKF process should be performed at $\omega_s \succsim 100\omega_c$ [13], with $\omega_s = 2\pi 200Hz$ to avoid the rocket crossover region, which is in frequencies around $\omega_c \approx 2\pi 2Hz$.

2.3 Optimal estimation theory

The system has continuous-time dynamics and discrete-time measurements [3]

$$\dot{x} = f(x, u, t) + w(t) \tag{I.44}$$

$$y_k = h(x_k, k) + v_k \tag{I.45}$$

where $w(t)$ is the process noise and v_k is the measurement noise, which are additive, uncorrelated, and white zero-mean Gaussian processes [3, 12]. Both processes satisfy [12] [12]

$$E\left[w(t)w^T(t+\tau)\right] = Q_e\delta(t) \qquad E\left[v_k v_{k+1}^T\right] = R_e\delta(k) \tag{I.46}$$

The estimation error $\varepsilon = x - \hat{x}$ is to be minimized in the cost function [12]

$$\min V = \lim_{t\to\infty} E\left[\varepsilon^T q q^T \varepsilon\right] \tag{I.47}$$

for which the solution is of the form [12]

$$V^* = q^T P q \tag{I.48}$$

In infinite time, P satisfies the filter algebraic Ricatti equation [12]

$$PF + F^T P - PH^T R_e^{-1} HP + Q_e = 0 \tag{I.49}$$

This results in the estimator feedback [3]

$$\tilde{x} = K_k\left[y - h(\hat{x}_k)\right] \tag{I.50}$$

$$K_k = P_k H_k^T R_e^{-1} \qquad K_k = P_k^- H_k^T\left[H_k P_k^- H_k^T + R_e\right]^{-1} \tag{I.51}$$

where the gain K_k is computed with the a-priori covariance estimate P_k^- (seen on the right side), as the current covariance P_k is dependent on the gain. Due to the nonlinear behaviour of the state space functions, both P and K_k must be computed in real time as new measurements become available [3]. It is to note that the Kalman filter generally is only optimal for the mathematical model, not the actual plant (in this case the rocket) itself [3]. Additionally, the EKF is just quasi-optimal, as optimality is generally only proven for the case of linear state prediction and measurement functions.

[12] Prof. Werner please forgive the butchering of the theory that is to follow

2.4 Filter tuning

As seen in Equation I.46, the tuning matrices Q_e and R_e represent the expected noise magnitudes of the state process and the measurements. Higher values in the matrices indicate a higher noise magnitude, i.e. lower trust in the accuracy of the measurement. The EKF is be tuned by adjusting the entries of these matrices, which must be positive semi-definite. The matrices can be filled to account for cross-covariance between the states or measurements (e.g. higher altitude or higher speed lowers accuracy of pressure measurement). However, this is often difficult to characterize, and the tuning matrices are often filled diagonally as initial guesses [13].

Typically, the values in R_e represent the noise behaviour of the sensors, where the the entries can be taken directly from sensor datasheets.

The matrix Q_e can be adjusted according to the expected "noise" in the state prediction process. Here, the entries can represent expected disturbances of the rocket flight, and again higher values indicate higher expected disturbances and thus lower trust in the accuracy of the state prediction.

2.5 Initialization routine

This algorithm serves to compute accurate values for $\bar{x}_0$ and $\bar{P}_0$. As the filter will be initialized while the rocket is stationary on the launch rail, several assumption can be made:

1. The velocity of the rocket v will be zero, and thus the altitude l is constant.
2. The acceleration of the rocket $\dot{v}$ will be zero.
3. The body rates ω will be zero, and thus the attitude q is constant.

Using the sensor models, this results in:

1. The accelerometer measurement will be the rotated gravity vector, so $S_S A = S^T g^G$. No bias can be set for the individual axes, but the magnitude of the accelerometer measurement should be g.
2. The gyroscope bias can set to the averaged measured angular velocity while the rocket is stationary.
3. Altitude can be set to altitude according to atmosphere model.
4. The initial orientation will be close to, but not exactly upright. The precise rotation can be determined by accelerometer and magnetometer measurements, for example with a complementary filter[13]. This resuts in the initial rotation angles / quaternion.

2.6 Hard calibration

The orientation of all sensor frames (S_Ω, S_A, S_M) to the rocket body must be determined *well before* launch day. Additionally, the position of the accelerometers r_S to the dry centner of gravity, and the the hard and soft iron biases for the magnetometer correction should be determined as well.

[13] Or Fast Accelerometer-Magnetometer Combination, or Madgwick, or TRIAD, or...

3 Controller

The controller is responsible for driving stabilizing the rocket state, attenuating any disturbances, and driving it to some desired value. In their configuration the canards can only provide a torque around the roll (x) axis of the rocket. This imposes some constraints on the controller design, as the whole state is under-actuated.

The controller chosen for the canards is linear state feedback, where the feedback gains are computed by the *Linear Quadratic Regulator* (LQR) method [13, 12]. The current state is determined by the state estimation algorithm (the Estimator), as seen in Figure I.3. The loop is a two-degree-of-freedom design, where the reference signal is prescaled by a reference gain. To account for the non-linearity of the rocket dynamics the gains are selected from a gain schedule, which is a look-up table of gains for different flight conditions.

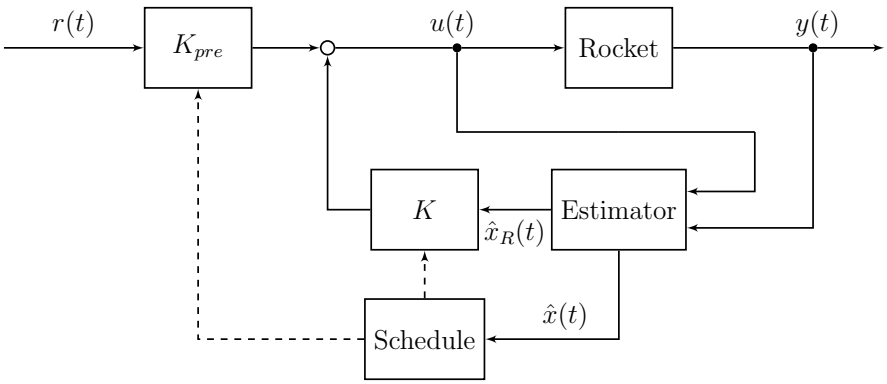


Figure I.3: Block diagram of the control loop. 2-DOF state estimate feedback [13], with gain scheduling.

3.1 Controller roll model

As the model must be linear, time-invariant, and controllable for the LQR synthesis, it makes some simplifications compared to Section 2. The cross coupling terms are neglected, and the the attitude is reduced from four quaternions to just the roll angle. This roll angle ϕ is computed from entries of the rotation matrix as $\phi = \text{atan2}(S_{23}, S_{33})$. The velocity and altitude show up in the dynamic pressure $\bar{p}$. The dynamic pressure and the canard coefficient C_L are the scheduling variables of the controller. The concocted state x_R for the roll model is

$$x_R = \begin{bmatrix} \phi & \omega_x & \delta \end{bmatrix}^T \tag{I.52}$$

The resulting state-space model is then

$$\dot{x}_R = Ax_R + B\delta_u \qquad\qquad y_R = Cx_R \tag{I.53}$$

with the matrices

$$A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & L_\delta \\ 0 & 0 & -\frac{1}{\tau} \end{bmatrix} \qquad B = \begin{bmatrix} 0 \\ 0 \\ 1/\tau \end{bmatrix} \qquad C = \begin{bmatrix} I_3 \end{bmatrix} \tag{I.54}$$

where the roll control derivative is $L_\delta(\bar{p}, C_L)$, a function of dynamic pressure and the canard coefficient of lift. The 1st order actuator dynamics are accounted for with the δ

state, and are characterized by the time constant τ (the value should be determined by experimental system identification). Any gear ratio between servo motor and canards may be expressed as the numerator of the bottom entry of B (here set to 1).

To determine if pole placement is possible, which is a necessary condition for LQR [13], the roll model is checked for controllability. The controllability matrix is

$$\mathcal{C} = \begin{bmatrix} B & AB & A^2B \end{bmatrix} \tag{I.55}$$

which has full rank for all $L_\delta \neq 0$, i.e. there exists a feasible feedback gain for every flight condition, except for zero velocity ($\|v\| = 0$) or a zero coefficient of lift ($C_L = 0$).

3.2 Feedback law and tuning

To determine the desired canard angle $u = \delta_u$, the the current rocket state as determined by the state estimation is used. The control law is a state feedback, where the gains K are determined by performing the *Linear Quadratic Regulator* (LQR) method around a linear model.

$$K(\hat{x}) = -\texttt{lqr}(A(\hat{x}), B, Q_c, R_c) \tag{I.56}$$
$$u(t) = K(\hat{x})\hat{x}_R(t) \tag{I.57}$$

The method is tuned by adjusting the weighting matrices Q_c and R_c. As shown in section 3.4, the matrix Q_c contains the weights for penalizing the state error, while the matrix R_c weighs the expended control effort.

3.3 Gain scheduling

As the models from canard angle to roll rate are nonlinear and highly dependent on some slowly-varying states, a gain-scheduling scheme will be implemented. The design points x_s of the schedule are characterized by these slowly-varying states, being as air density, velocity, and aerodynamic coefficients of the canards. The air density and velocity are combined to the dynamic pressure $\bar{p}$. The simplified model is evaluated around these design points using the current state estimate (so $A(\hat{x})$), to form the matrix A. As the LQR method produces guaranteed robustness margins [14, 13, 12], the interpolation between two close design points should always produce a stable control loop.

A large variety of design points are chosen, both along the nominal flight profile, but also encompassing a larger flight envelope to accommodate off-nominal trajectories. Design points for the condition L_δ are omitted from the tuning process. The control gains for all design points are computed beforehand and stored in a look-up table. In flight, the control algorithms interpolate the gains from the look-up table according to the current state estimate. The pre-computed feedback gains for the roll model are shown in Figure I.4, plotted over the scheduling variables $\bar{p}$ and C_L. To note is the discontinuity at $\bar{p} = 0$ and especially at $C_L = 0$, where no feasible design points exist. The feedback gain are interpolated over this split.

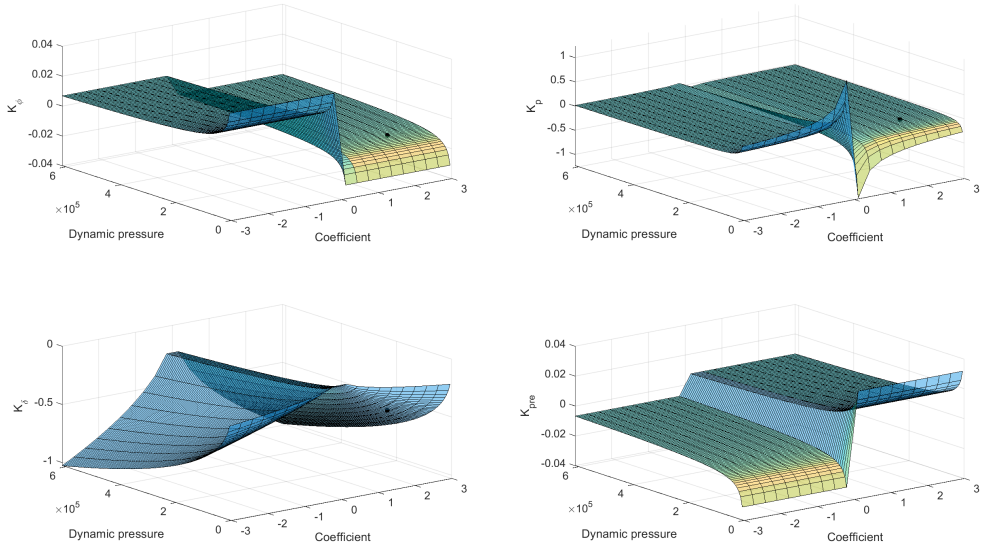


Figure I.4: Scheduling surfaces along dynamic pressure $\bar{p}$ and canard coefficient of lift C_L. Feedback gains K_ϕ, K_p, K_δ and reference gain K_{pre}.

3.4 Optimal control theory

The LQR method uses the weighting matrices Q and R to solve the linear quadratic optimization problem [12] [14]

$$\min V = \int_0^\infty (x^T(t)Qx(t) + u^T(t)Ru(t))dt \tag{I.58}$$

for which the solution is of the form

$$V^* = x^T P x \tag{I.59}$$

In infinite time, P satisfies the algebraic Ricatti equation

$$PF + F^T P - PBR^{-1}B^T P + Q = 0 \tag{I.60}$$

which results in the control law

$$u^*(t) = Kx(t) \tag{I.61}$$

$$K = -R^{-1}B^T P \tag{I.62}$$

The resulting controller for the linear system is optimal in the trade-off of minimizing the state error $x(t)$ and control effort $u(t)$ [13]. The closed-loop is guaranteed to be stable [12], and has margins of $[\infty, 0.5]$ in gain and $60°$ in phase. This guarantee is only valid however for linear systems with full state measurements [14]. The control law is still useful for controlling nonlinear and observed systems, as the robustness properties can be partially preserved [12].

[14] I think this came out better than the estimation theory

4 Appendix

4.1 Numerical Jacobian computation

The Jacobian J_x is the matrix of all first-order partial derivatives of a vector-valued function. For the prediction and measurement model, the Jacobians of the functions f and h ([15]) are needed with respect to the state x (where $n = \text{length}(x)$). The Jacobian of any function with respect to x is defined as [15]

$$J_x = \frac{\partial f}{\partial x} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix} \tag{I.63}$$

Jacobians may be derived analytically from the function, but this is often cumbersome and prone to errors. Therefore, it is common practice to compute the matrix numerically. A frequently used method is using *finite differences*, however this is known to be not very efficient nor accurate, and has issues with numerical instability [15].

Complex-step differentiation

A more efficient way to calculate Jacobian matrices numerically is to use *complex step differentiation*. In this method with step size ε, the k-th column of the matrix (J_{x_k}) is [16]

$$J_{x_k} = \text{Im}\left(\frac{f(t, x + \mathrm{i}\varepsilon e_k, u)}{\varepsilon}\right) \tag{I.64}$$

where e_k is the k-th unit vector in the state-space (i.e. $e \in X$, and $e_1 = \begin{bmatrix} 1 & 0 & \cdots & 0 \end{bmatrix}^T$, $e_2 = \begin{bmatrix} 0 & 1 & \cdots & 0 \end{bmatrix}^T$, etc.). This must be computed for each entry of x, so that the Jacobian is

$$J_x = \begin{bmatrix} J_{x_1} & J_{x_2} & \cdots & J_{x_n} \end{bmatrix} \tag{I.65}$$

The evaluation of the functions with a complex state vector $\begin{bmatrix} x_1 + \mathrm{i} & x_2 & \cdots & x_n \end{bmatrix}^T$ seems very unintuitive, but works for every holomorphic $\mathbb{R}^n \to \mathbb{R}^m$ function[16] [15]. Complex-step differentiation also is numerically stable, and has also an $\mathrm{O}(h^2)$ error size [16]. Both finite differences and complex-step scale linearly with the number of variables, but the finite difference method requires two function evaluations per variable, while complex-step requires only one.

4.2 Numerical Integration methods

To get a new discrete state from the continuous-time prediction process of the EKF, the nonlinear differential equations I.37 and I.38 have to be solved. As the last state estimate is available as an initial value, this can be expressed as the *Initial Value Problem* (IVP)

$$\dot{x}(t) = f(t, x, u) \qquad\qquad x(0) = x_n \tag{I.66}$$

which is can be solved by numerical integration. The integration methods need to be explicit, as the filter in the prediction step has only past state information.

Step size ε

[15] The following equations are shown for an "f", but are of course the same for f and h

[16] for implementation this requires that the function can be evaluated with complex inputs

Explicit Runge-Kutta

The explicit s-stage *Runge-Kutta* method is compactly written as [17]

$$x_{n+1} = x_n + \varepsilon_n \sum_{i=1}^{s} c_i k_i \tag{I.67}$$

$$k_i = f\left(t_n + a_i \varepsilon_n\,,\; x_n + \varepsilon_n \sum_{j=1}^{i-1} b_{ij} k_j\,,\; u\right) \tag{I.68}$$

where a_i, b_{ij}, and c_i are the entries of the vectors a and c and the matrix B, displayed in a *Butcher tableau*

$$\begin{array}{c|c} a & B \\ \hline & c^T \end{array} \tag{I.69}$$

These can be very large, as in the *Dormand-Prince* method (RK45, which the standard solver in Matlab). For our cases we will keep the tableaus small.

The second order ($s = 2$) butcher tableau of the Heun method [17], which is used for the prediction covariance, is

$$\begin{array}{c|cc} 0 & & \\ 1 & 1 & \\ \hline & \frac{1}{2} & \frac{1}{2} \end{array} \tag{I.70}$$

The fourth order ($s = 4$) butcher tableau of the classical Runge-Kutta method RK4 [17], which is used for solving the state prediction, is

$$\begin{array}{c|cccc} 0 & & & & \\ \frac{1}{2} & \frac{1}{2} & & & \\ \frac{1}{2} & 0 & \frac{1}{2} & & \\ 1 & 0 & 0 & 1 & \\ \hline & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6} \end{array} \tag{I.71}$$

5 Index

Short	Expansion	Description
EKF	Extended Kalman Filter	Extension of Kalman filter to nonlinear systems
IMU	Inertial Measurement Unit	Packaged gyroscope and accelerometer sensor
IVP	Initial Value Problem	ODE set to be solved, with inital values
LQR	Linear Quadratic Regulator	Optimal controller for linear systems

Table I.2: Acronyms

Name	Dimension	Description
x	$n\times1$	State
$\hat{x}$	$n\times1$	State estimate
u	$m\times1$	Control input (controller $\rightarrow$ rocket)
y	$l\times1$	Measurement output (rocket $\rightarrow$ estimator)
$f(x,u)$	$n\times1$ function	State transition function
$h(x)$	$l\times1$ function	Measurement function
$F(x,u)$	$n\times n$ function	State transition Jacobian
$H(x)$	$l\times n$ function	Measurement Jacobian
$B(x,u)$	$n\times m$ function	Input Jacobian
P	$n\times n$	Covariance estimate
K_k	$l\times n$	Filter gain
Q_e	$n\times n$	State process covariance
R_e	$l\times l$	Measurement covariance
K_s	$m\times n$	Feedback gain
Q_c	$n\times n$	State error weighting
R_c	$m\times m$	Control effort weighting

Table I.3: Estimator and Controller variables

Name	Dimension	Unit	Description
q	4		Quaternion
ω	3	$\frac{\text{rad}}{\text{s}}$	Angular velocity in body-fixed frame (body rates)
v	3	$\frac{\text{m}}{\text{s}}$	Velocity in the body-fixed frame
r^G	3	m	Position in the inertial (geographic) frame
l	1	m	Altitude
S	3×3		Rotation matrix, body-fixed $\rightarrow$ inertial
S_i	3×3		Rotation matrix, sensor $i \rightarrow$ body-fixed
Ω	3	$\frac{\text{rad}}{\text{s}}$	Measured body rates
M	3	mT	Measured magnetic field
A	3	$\frac{\text{m}}{\text{s}^2}$	Measured acceleration
P	1	Pa	Measured air pressure
K	1	K	Measured air temperature
m		kg	Mass
J	3×3	kgm^2	Moment of inertia
$\mathcal{F}$	3	N	External force in the body-fixed frame
$\mathcal{T}$	3	Nm	External torque in the body-fixed frame
g		$\frac{\text{m}}{\text{s}^2}$	Gravitational acceleration
V_A		$\frac{\text{m}}{\text{s}}$	Airspeed
T		K	Air temperature
ρ		$\frac{\text{kg}}{\text{m}^3}$	Air density
p		Pa	Static pressure
$\bar{p}$		Pa	Dynamic pressure

Table I.4: Important modelling variables

Chapter II

Rocket flight Simulation

Joe Dolina, Tristan Yan-Klassen

6 Plant Model

6.1 Background

2023-2024, Borealis, and Airbrakes

Borealis was the team's first attempt at flying (and for all intents and purposes, developing) active control systems, in the form of airbrakes. At the outset of the airbrakes project, it was envisioned that a working group would develop something like Fig II.1. Incidentally, it was "envisioned" that we would be flying TVC stabilization in 2026.

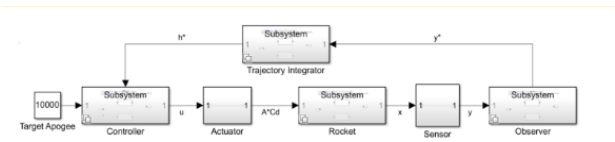


Figure II.1: Airbrakes Simulink structure

Everything except "Actuator" and "Rocket" were implemented to some degree in the final flight software. However, the most useful output from Simulink that year was Fig II.1 itself.

A PID controller was ultimately selected for airbrakes because nobody had the knowledge or experience to do anything fancier. This controller was "tuned" using a plugin for OpenRocket [18], essentially going in and dynamically modifying the drag force used by Openrocket in its calculations with values produced from external computational fluid dynamics (CFD) simulations. Later versions included the ability to inject random noise into the state measurements. Under these noisy conditions, simulations suggested the controller could get within about 20m of the target altitude over most of the controllable regime (20,000 +/- 2,000 ft.).

While much more useful at the time than having no controller-in-loop simulation options at all, the OpenRocket plugin was not sustainable. The overlap between Waterloo Rocketry members (and budding control engineers) and Java developers was and still is very small.

The final version ran the controller as a binary compiled from the same C source as the board on the rocket, and while this was also a promising development in the team's testing practices, the process for doing this was even less accessible than running the plugin itself. The plugin project was ultimately an uphill battle to turn OpenRocket into something it wasn't meant to be.

This brings us to the present design cycle. The controls subsystem has flight heritage in the form of the processor and motor control boards, it has experienced members (in this author's opinion, some of the very best members), and it even has its own WEEF account. Given the sophistication of controller and state estimator design we are targeting this year for canards, as well as the broad interest from controls members in Matlab and Simulink, there is a strong justification to pursue the development of a complete rocket plant model in Simulink this year.

Project Jupiter

Projeto Jupiter is a rocketry team from the University of São Paulo, Brazil. They conveniently already developed most of what is necessary for a plant model of a sounding rocket with canards [19]. Rather than trying to start more or less from scratch as it was planned last year, the goal this year is essentially to modify their plant model with our rocket design parameters, verify it for uncontrolled flight, and then begin simulating the effects of canards.

6.2 Theory

Compared to a dynamic model of a real launch vehicle, a sounding rocket is considerably simpler. In particular, sloshing (source: Thomas), bending, and TVC dynamics do not need to be considered. The rocket can be treated as a single rigid body with time-varying mass and mass distribution. The only forces acting on an uncontrolled rocket are engine thrust, gravity, and drag. Of these, gravity applies no moment by definition, and the moment applied by thrust, due to misalignment with the rocket center of mass, is considered negligible for our purposes. Since gravity is well defined and engine thrust is defined externally from static fire test data, this leaves aerodynamic drag as the most complicated aspect of simulation.

Barrowman provides empirical methods for computing the center of pressure of a rocket's components at relatively low Mach numbers. (link FD docs). The coefficient of drag (C_d) is modeled only as a function of Mach number for a given rocket, using OpenRocket's methodology, of which this author is unfamiliar. In the future, other drag models could certainly be considered.

6.3 Implementation

Everything is in metric - MKS. Due to restrictions of certain blocks, the WGS84 earth model is used where applicable.

Matlab Scripts

A rocket-specific script (e.g. Borealis.m) hard-codes the geometric parameters like reference length, fin location, etc. This script also imports a .csv file from OpenRocket containing

relevant data. The Borealis script includes some logic to turn the C_d and Mach number time series into a lookup table. Barrowman.m takes the rocket parameters and evaluates the Barrowman equations based on the reference geometry.

Top Level Plant

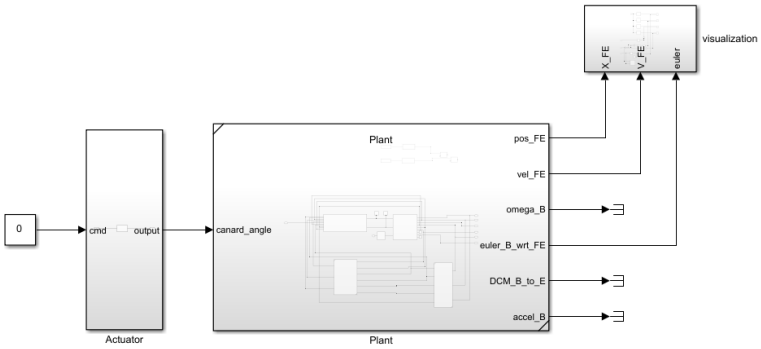


Figure II.2: Canards Plant Model Interface

The only explicit input to the plant is the deflection angle of the canards. The plant model outputs the full 6DOF state in a combination of reference frames, and the direction cosine matrix (DCM) from flat earth to body frame. It also outputs the translational acceleration, in order to model accelerometer dynamics.

The actuator subsystem implements a first order system representing the response delay of the servo.

6DOF Variant Mass Model

The core of the plant model is the "Simple Variable Mass 6DOF" block provided by the Aerospace Blockset. It takes in net force and moment about the center of mass at each time step and outputs translational and rotational position and velocity. Note that the inputs are provided in the body fixed frame, with the X axis coincident with the longitudinal axis of the rocket body. The translational position and velocity are provided in the flat earth reference frame. The block doesn't output mass vs time, instead giving you starting mass, ending mass, and a $\frac{dm}{dt}$ input. You gotta dig in there and log the signal or hope and pray. But it seems to work as expected so far.

Wind Model

The wind simulation operates similar to Niskanen. The winds take on a constant speed at all altitudes, with pink noise for turbulence added on top. This turbulence is also responsible for any and all rotational speed of the wind. Additionally, gusting has been added on top of the base wind model, which allows for gusts of set strength and direction to happen after a specified time. Similar to OpenRocket, the simulation takes a user inputted base wind velocity, base wind direction, and turbulence value. The user must also specify the magnitude, time of occurrence, and duration of gusts.

Roll Dynamics

For roll dynamics, the canards and fins are split into separate parts and treated independently. This allows the canards to be handled with more detail or with different aerodynamic models if needed.

For the fins, the aerodynamic coefficients and derivatives are calculated using Barrowman's equations and then fed into the Simulink model. This is done by the "roll forcing and damping block" in the plant. The only significant change from an aerodynamics standpoint is the mach-speed correction term on the coefficients, which is normally taken to be

$$\mathcal{B}^{-1} = \frac{1}{\sqrt{|1 - M^2|}}$$

Since this breaks at $M = 0$ and it is commonly held that it peaks at around 2 near sonic speeds, we take $\mathcal{B}$ to be the minimum of the above expression and 2.

Normal Force Components

The methodology used to compute the normal force components is the same as that of OpenRocket [20] (see Sec. 3.2.1). The rocket body is divided into its components: nosecone, body, fin, tail, and canards (if applicable). The local velocity of the rocket at the component center of pressure, relative to the wind, is computed and along with the component C_{N_α} is used to compute the normal force and moment vectors. This functionality is encapsulated in a generic "lift component" block, whose internals are shown in Fig II.3:

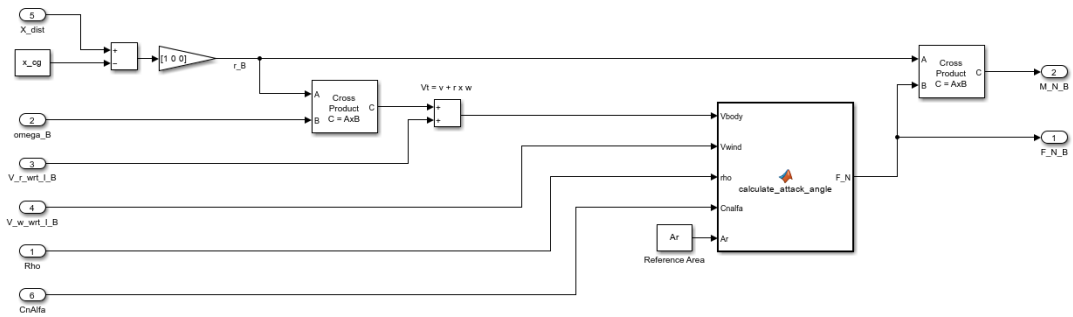


Figure II.3: Lift Component subsystem

These blocks are composed to calculate the net normal force and moment:

Simulated Sensor Outputs

In order to test the robustness of the controller to real-world sensor outputs, IMU (accelerometer, gyroscope, magnetometer) and barometer (pressure, temperature) outputs are simulated, adding noise and drift. This is done through custom MatLab scripts that track drift over the simulation and continuously add it to the measured quantities.

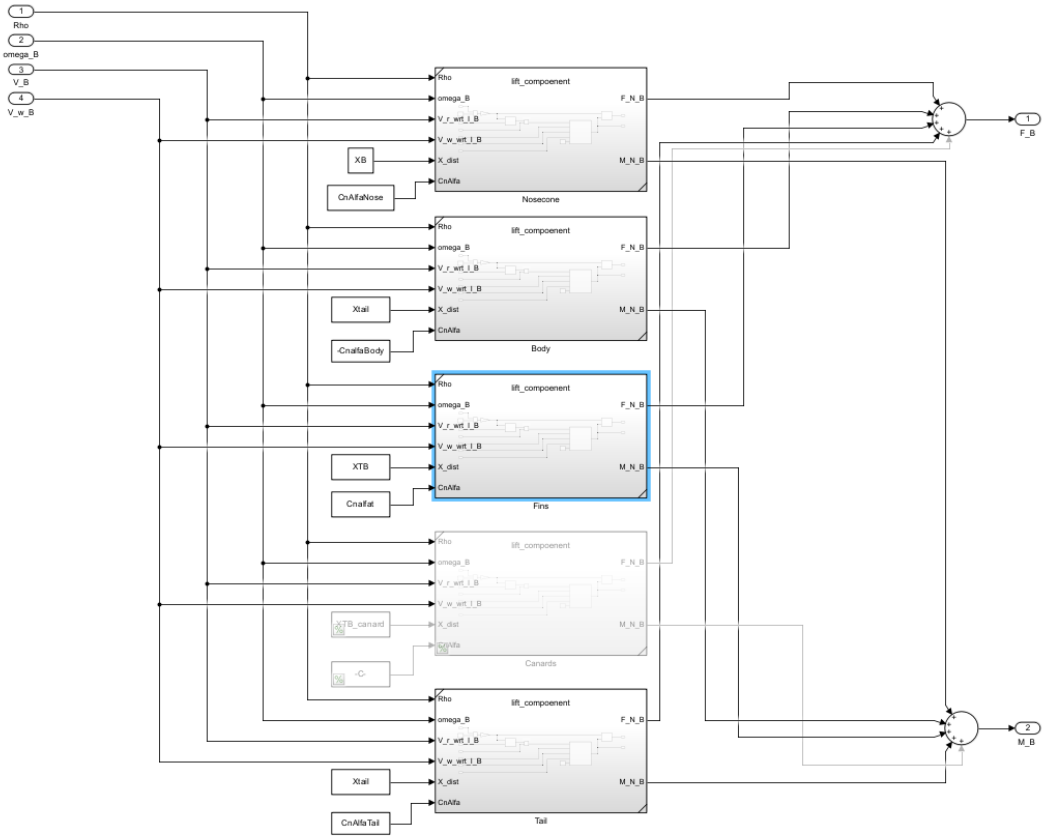


Figure II.4: Normal Force and Moment

7 Simulation results

7.1 Rocket flight

Comparison with OpenRocket

7.2 Estimator in the loop

7.3 Controller in the loop

7.4 Full closed-loop

References

[1] Peter H. Zipfel. *Modeling and Simulation of Aerospace Vehicle Dynamics*. 2nd. Reston: American Institute of Aeronautics and Astronautics, 2007. ISBN: 978-1-56347-875-8. DOI: 10.2514/4.862182.

[2] Brian L. Stevens, Eric N. Johnson, and Frank L. Lewis. *Aircraft Control and Simulation: Dynamics, Controls Design, and Autonomous Systems*. Third edition. Hoboken, New Jersey: Wiley, 2015. ISBN: 9781119174882. DOI: 10.1002/9781119174882.

[3] Frank L. Lewis, Lihua Xie, and Dan Popa. *Optimal and Robust Estimation*. 2nd. CRC Press, Taylor & Francis Group, 2008. ISBN: 978-0-8493-9008-1.

[4] Ashish Tewari. *Advanced control of aircraft, spacecraft and rockets*. Aerospace series. Hoboken, N.J: Wiley, 2011. ISBN: 1119971209. DOI: 10.1002/9781119971191.

[5] Julian Theis. *Flight Control Law Design: Lecture Notes*. Hamburg University of Technology, Institute of Aircraft Systems (FST), 2023.

[6] Bryan Tong Minh. "Real-time position and attitude determination of the Stratos II sounding rocket". MA thesis. Delft University of Technology, 2012.

[7] Julian Theis et al. "Linear Parameter-Varying Feedforward Control: A Missile Autopilot Design". In: *AIAA Guidance, Navigation, and Control Conference 2015*. Red Hook, NY: Curran, 2015. ISBN: 978-1-62410-339-1. DOI: 10.2514/6.2015-2001.

[8] Werner Schiehlen and Peter Eberhard. *Technische Dynamik*. 5th. Springer Vieweg, 2017. ISBN: 978-3-658-18456-8. DOI: 10.1007/978-3-658-18457-5.

[9] Robert F. Stengel. *Flight Dynamics*. Princeton, NJ: Princeton University Press, 2004. ISBN: 9781400866816. DOI: 10.1515/9781400866816.

[10] James S. Barrowman. "The practical calculation of the aerodynamic characteristics of slender finned vehicles". M.S. USA – Washington, D.C.: The Catholic University of America, 1967. URL: https://ntrs.nasa.gov/api/citations/20010047838/downloads/20010047838.pdf.

[11] *VectorNav Inertial Navigation Primer*. 2024. URL: https://www.vectornav.com/resources/inertial-navigation-primer/ (visited on 12/07/2024).

[12] Herbert Werner. *Optimal and Robust Control: Lecture Notes*. Hamburg University of Technology, Institute of Control Systems, 2021.

[13] Herbert Werner. *Control Systems Theory and Design: Lecture Notes*. Hamburg University of Technology, Institute of Control Systems, 2021.

[14] J. Doyle. "Guaranteed margins for LQG regulators". In: *IEEE Transactions on Automatic Control* 23.4 (1978), pp. 756–757. ISSN: 0018-9286. DOI: 10.1109/TAC.1978.1101812.

[15] Joaquim R. R. A. Martins and Andrew Ning. "Engineering Design Optimization". In: (2021).

[16] Joaquim R. R. A. Martins, Peter Sturdza, and Juan J. Alonso. "The complex-step derivative approximation". In: *ACM Trans. Math. Softw.* 29.3 (Sept. 2003), pp. 245–262. ISSN: 0098-3500. DOI: 10.1145/838250.838251. URL: https://doi.org/10.1145/838250.838251.

[17] David Griffiths and Desmond Higham. *Numerical Methods for Ordinary Differential Equations: Initial Value Problems*. Jan. 2010. ISBN: 978-0-85729-147-9. DOI: 10.1007/978-0-85729-148-6.

[18] *OR Airbrake Plugin*. https://github.com/waterloo-rocketry/or-airbrake-plugin.

[19] *Project Jupiter Control Canards*. https://github.com/Projeto-Jupiter/CC.

[20] *OpenRocket technical documentation v13.05*. https://openrocket.sourceforge.net/techdoc.pdf.